

计算机组成与体系结构课内实践 #2

提交时间: 23:59, 7 月 12 日, 2026

河海大学计算机与软件学院

重要提醒: 本次课内实践作业是本课程第一次涉及流水线数据通路设计的综合性实践, 整体难度明显高于第一次作业。作业不仅要求完成 Verilog 代码编写, 还要求完成测试平台 (Testbench) 设计、波形验证以及实验报告撰写。其中, 题目二需要搭建简化五级流水线数据通路, 题目三涉及数据转发 (Forwarding) 和 Load-Use 冒险检测 (Hazard Detection), 这些内容都是计算机组成原理课程中的重点和难点。

请大家务必尽早开始, 不要等到截止日期前一两天再动手。即使借助 AI 工具生成部分代码, 仍然需要自己理解设计思路、完成模块集成、编写测试平台并分析仿真波形。代码编写往往只占整个作业工作量的一部分, 测试与调试才是最耗费时间的环节。若前期没有认真学习相关知识, 仅依赖 AI 工具在短时间内完成整个作业是非常困难的。

另外, 本课程的平时成绩占总评成绩的 30%, 课内实践作业是平时成绩的重要组成部分。若作业不完成、不提交或提交质量过低, 将对平时成绩造成较大影响, 甚至可能使总评成绩处于**不及格边缘**。请大家高度重视本次实践作业, 按时完成并认真提交。

声明: 在本次作业中请做到以下几点:

- 请将学号和姓名清晰的写在作业第一页, 请给出详细的注释, 严禁抄袭。
- 作业需提交源码、测试代码、运行结果和实验报告。请将所有提交材料打包为一个 zip 附件, 并通过电子邮件发送给任课教师。附件命名格式必须为“**学号-姓名-计组实践作业 2.zip**”。zip 附件中应包含 PDF 格式的实验报告、Verilog 源码文件和测试代码文件。运行结果应以截图形式放入 PDF 实验报告中, 并对关键仿真结果进行说明。请注意, 实验报告只接受 PDF 格式, 源码文件和测试代码文件必须为*.v文件, 压缩包只接受 zip 格式, 不接受其他压缩文件格式。
- 请确保**电子邮件标题**严格遵循以下格式: “**学号-姓名-计组实践作业 2**”。若因邮件标题或附件命名不规范导致作业无法及时定位, 可能影响成绩记录。
- **本次作业不接受迟交, 若因客观原因无法完成, 请提前 24 小时联系老师并说明情况。**

计算机体系结构：Verilog 流水线 CPU 数据通路设计

学习目标

- 理解五级流水线的基本结构，包括 IF、ID、EX、MEM、WB 五个阶段；
- 掌握流水线寄存器的设计方法，理解级间寄存器在流水线中的作用；
- 理解流水线中的 stall、flush 和 valid 信号；
- 理解数据冒险、load-use 冒险和转发机制；
- 能够使用 Verilog 实现简化的流水线数据通路，并通过 testbench 观察每个周期的数据流动。

题目一：带暂停和清空功能的流水线寄存器设计

请设计一个通用流水线寄存器模块。该模块用于连接流水线的相邻阶段，例如 IF/ID、ID/EX、EX/MEM、MEM/WB。流水线寄存器需要支持正常传递、暂停保持、清空插入气泡和复位清零。

模块接口如下：

```
1 module pipe_reg #(
2     parameter WIDTH = 32
3 )(
4     input wire clk,
5     input wire rst_n,
6     input wire stall,
7     input wire flush,
8     input wire [WIDTH-1:0] din,
9     output reg [WIDTH-1:0] dout
10 );
```

Listing 1: 通用流水线寄存器模块接口

设计要求如下：

- `rst_n` 为异步、低电平有效复位，优先级最高：当 `rst_n = 0` 时，`dout` 立即清零，与 `clk`、`stall`、`flush` 的取值无关；
- 在 `rst_n = 1` 的前提下，`flush` 和 `stall` 均为同步信号，仅在 `clk` 上升沿生效；
- 当 `flush = 1` 时，在时钟上升沿将 `dout` 清零，表示向流水线中插入一个气泡；

- 当 `stall = 1` 且 `flush = 0` 时, `dout` 保持原值不变;
- 当 `stall = 0` 且 `flush = 0` 时, 在时钟上升沿将 `din` 传递到 `dout`;
- 若 `stall` 和 `flush` 同时为 1, 规定 `flush` 优先, `dout` 清零;
- 三者优先级由高到低为: `rst_n` (异步复位) > `flush` > `stall`;
- 要求使用 `always @(posedge clk or negedge rst_n)` 描述时序逻辑。

设计说明:

- “插入气泡”在本题中等价于将 `dout` 清零。当本模块用于保存控制信号 (如 `op_type`) 时, 清零即得到 NOP 编码, 从而在流水线中形成一个不产生副作用的空操作。这一点将在题目三的冒险处理中用到。

测试要求如下:

1. 复位后检查 `dout` 是否为 0;
2. 在正常模式下连续输入不同数据, 验证 `dout` 是否按周期更新;
3. 设置 `stall = 1`, 验证 `dout` 是否保持**前一拍的非零值**不变;
4. 在 `dout` 为非零值的前提下设置 `flush = 1`, 验证 `dout` 是否被清零 (以区别于 `stall` 保持);
5. 同时设置 `stall = 1` 和 `flush = 1`, 且使前一拍 `dout` 为非零值, 验证 `flush` 是否优先 (`dout` 应清零而非保持)。

报告要求如下:

- 说明流水线寄存器在五级流水线中的作用;
- 解释 `stall` 和 `flush` 的区别;
- 给出仿真波形, 并标注正常传递、暂停保持和清空气泡对应的周期;
- 说明为什么 `flush` 通常需要高于 `stall` 的优先级;
- 说明为什么异步复位 `rst_n` 的优先级应高于 `flush` 和 `stall`。

题目二：简化五级流水线数据通路设计

请在第一次实践作业中 ALU、寄存器堆和数据存储器的基础上，设计一个简化的五级流水线数据通路。本题不要求实现完整指令译码器，也不要求支持完整指令集，而是通过外部输入的控制字段模拟已经译码好的指令，使大家重点理解五级流水线中数据如何逐级传递。**注意：题目二暂不要求实现数据冒险检测和转发机制。**因此，本题中的测试序列应避免连续数据相关；若后一条指令需要使用前一条指令的结果，应在两条指令之间显式插入足够的 NOP，使前一条指令完成写回后再使用其结果。数据冒险的自动检测、转发和暂停机制将在题目三中完成。

本题中的五级流水线为：

$$\text{IF} \rightarrow \text{ID} \rightarrow \text{EX} \rightarrow \text{MEM} \rightarrow \text{WB}$$

其中：

- IF 阶段：接收一条外部输入的简化指令包；
- ID 阶段：读取寄存器堆，获得源操作数；
- EX 阶段：执行 ALU 运算或计算访存地址；
- MEM 阶段：访问数据存储器；
- WB 阶段：将结果写回寄存器堆。

关键时序约定（务必遵守，否则期望结果不可复现）：

- **寄存器堆采用“写优先 (write-first)”同步写、组合读模型：**WB 阶段在 `clk` 上升沿写入寄存器堆；ID 阶段的读取为组合逻辑（异步读）。当同一周期内 WB 写入的目的寄存器与 ID 读取的源寄存器相同时，**ID 应读到本周期 WB 写入的新值**。这一约定保证了在插入足够 NOP 后结果唯一确定。
- 由于题目二要求显式插入 NOP 以彻底分离相关指令，正常情况下不会触发上述同周期读写；该约定主要用于消除边界周期的歧义。
- 数据存储器同样采用同步写、组合读模型，写发生在 MEM 阶段的 `clk` 上升沿。

简化指令包格式如下：

字段	位宽	含义
valid	1 位	当前输入是否为有效指令
op_type	2 位	操作类型
rs1	3 位	源寄存器 1
rs2	3 位	源寄存器 2
rd	3 位	目的寄存器
imm	16 位	立即数
alu_op	3 位	ALU 操作码
byte_en	2 位	字节写使能

op_type 定义如下:

op_type	操作类型	功能描述
2'b00	NOP	空操作, 不写寄存器, 不写存储器
2'b01	ALU	$rd = rs1 \text{ op } rs2$
2'b10	LOAD	$rd = Mem[rs1 + imm]$
2'b11	STORE	$Mem[rs1 + imm] = rs2$

alu_op 编码如下:

ALU 操作	编码
加法	3'b000
减法	3'b001
按位与	3'b010
按位或	3'b011

byte_en 字节写使能定义如下:

数据通路为 16 位 (两字节)。byte_en 仅在 STORE 操作时有效, 按位控制写入数据存储器的字节通道, 高电平有效:

byte_en	含义	对 Mem[addr] 的影响
2'b00	不写入	保持原值 (NOP/ALU/LOAD 时取此值)
2'b01	仅写低字节	仅更新 [7:0], [15:8] 保持
2'b10	仅写高字节	仅更新 [15:8], [7:0] 保持
2'b11	写入整字	同时更新 [15:8] 和 [7:0]

其中 byte_en[0] 对应低字节 [7:0], byte_en[1] 对应高字节 [15:8]。本题测试中 STORE 均使用 2'b11 (整字写入)。

模块接口如下:

```

1 module simple_pipeline_cpu (
2     input wire clk,
3     input wire rst_n,
4
5     input wire instr_valid,
6     input wire [1:0] instr_op_type,
7     input wire [2:0] instr_rs1,
8     input wire [2:0] instr_rs2,
9     input wire [2:0] instr_rd,
10    input wire [15:0] instr_imm,
11    input wire [2:0] instr_alu_op,
12    input wire [1:0] instr_byte_en,
13
14    input wire init_we,
15    input wire [2:0] init_addr,
16    input wire [15:0] init_data,
17
18    input wire [2:0] debug_reg_addr,
19    output wire [15:0] debug_reg_data,
20    input wire [4:0] debug_mem_addr,
21    output wire [15:0] debug_mem_data,
22
23    output wire [15:0] debug_if_id_imm,
24    output wire [15:0] debug_id_ex_src_a,
25    output wire [15:0] debug_id_ex_src_b,
26    output wire [15:0] debug_ex_mem_alu_result,
27    output wire [15:0] debug_mem_wb_wb_data
28
29 );

```

Listing 2: 简化五级流水线数据通路模块接口

调试接口说明如下：

- debug_reg_addr 和 debug_reg_data 用于在 testbench 中读取寄存器堆中指定寄存器的最终值；
- debug_mem_addr 和 debug_mem_data 用于在 testbench 中读取数据存储器中指定地址的最终值；
- debug_if_id_imm、debug_id_ex_src_a、debug_id_ex_src_b、debug_ex_mem_alu_result 和 debug_mem_wb_wb_data 用于观察指令和数据在流水线各阶段之间的传递过程。

内部模块要求如下：

- 内部包含一个 8x16 寄存器堆；
- 内部包含一个 16 位 ALU；
- 内部包含一个 32x16 数据存储器；
- 内部至少包含四组流水线寄存器：
 1. IF/ID 流水线寄存器；
 2. ID/EX 流水线寄存器；
 3. EX/MEM 流水线寄存器；
 4. MEM/WB 流水线寄存器。
- 各级之间传递的不仅包括数据，还应包括必要的控制信号，例如 `valid`、`op_type`、`rd`、`alu_op`、`byte_en` 等；
- `init_we` 用于测试时初始化寄存器堆，优先级高于正常 WB 阶段写回；
- R0 规定为常零寄存器，写入 R0 无效，读取 R0 恒为 0；
- 数据存储器访问地址统一取 `alu_result[4:0]`（低 5 位），以匹配 32 深度存储器，超出部分被截断；
- 题目二不要求实现数据转发、冒险检测或自动暂停。

各阶段功能要求如下：

1. IF 阶段

- 接收外部输入的简化指令字段；
- 当 `instr_valid = 1` 时，认为输入为有效指令；
- 将指令字段写入 IF/ID 流水线寄存器；
- 当 `instr_valid = 0` 时，将该周期输入视为 NOP；
- NOP 指令不得写寄存器，也不得写数据存储器。

2. ID 阶段

- 根据 IF/ID 阶段保存的 `rs1` 和 `rs2` 读取寄存器堆（组合读）；
- 将读出的 `src_a` 和 `src_b` 以及其他控制信号写入 ID/EX 流水线寄存器；
- 对于 LOAD 和 STORE, `src_a` 作为基地址, `imm` 作为偏移量；
- 对于 ALU 操作, `src_a` 和 `src_b` 作为 ALU 操作数。

3. EX 阶段

- 对于 ALU 操作, 计算 `src_a op src_b`；
- 对于 LOAD 和 STORE, 计算访存地址 `src_a + imm`；
- ALU 结果和必要控制信号写入 EX/MEM 流水线寄存器；
- STORE 操作还需要继续向后传递 `src_b`, 因为 `src_b` 是要写入内存的数据。

4. MEM 阶段

- 对于 LOAD 操作, 从数据存储器读出数据；
- 对于 STORE 操作, 按 `byte_en` 将 `src_b` 写入数据存储器；
- 对于 ALU 操作, 不访问存储器, 只将 ALU 结果继续传递；
- 数据存储器地址使用 `alu_result[4:0]`；
- STORE 操作的字节写使能由随流水线传递到 MEM 阶段的 `byte_en` 决定。

5. WB 阶段

- 对于 ALU 操作, 将 ALU 结果写回 `rd`；
- 对于 LOAD 操作, 将存储器读出的数据写回 `rd`；
- 对于 STORE 和 NOP 操作, 不写回寄存器；
- 写回寄存器堆时需要判断 `rd != 0`（写入 R0 无效）；
- 写回数据应输出到 `debug_mem_wb_wb_data`, 便于在波形中观察。

测试要求如下：

请在 testbench 中至少完成以下指令序列。为了便于观察，每个周期输入一条简化指令；当指令之间存在数据相关时，应显式插入 NOP，使前一条指令完成写回后再使用其结果。

1. 复位系统；

2. 使用 init_we 初始化寄存器：

- $R1 = 16'h0004$
- $R2 = 16'h0006$
- $R3 = 16'h1234$
- $R7 = 16'h00FF$

3. 输入 ALU 指令： $R4 = R1 + R2$ ，期望最终 $R4 = 16'h000A$ ；

4. 输入 STORE 指令： $Mem[R1 + 0] = R3$ ，期望最终 $Mem[4] = 16'h1234$ ；

5. 输入 LOAD 指令： $R5 = Mem[R1 + 0]$ ，期望最终 $R5 = 16'h1234$ ；

6. 连续输入若干 NOP，使 R4 完成 WB 阶段并彻底退出流水线；

7. 输入 ALU 指令： $R6 = R4 - R2$ ，期望最终 $R6 = 16'h0004$ ；

8. 继续输入若干 NOP，使所有指令完成 WB 阶段；

9. 通过仿真波形观察每条指令在 IF、ID、EX、MEM、WB 中的移动过程；

10. 使用 debug_reg_addr 和 debug_mem_addr 检查最终寄存器和数据存储器内容。

注意：由于这是五级流水线，输入指令后不会立即得到最终结果。学生需要继续输入若干个 NOP，使流水线中的指令完成后续阶段。检查最终结果时，应至少等待最后一条有效指令进入 WB 阶段之后再观察。

报告要求如下：

- 画出五级流水线数据通路图；
- 用表格列出每条测试指令在每个周期所处的流水线阶段；
- 说明 IF/ID、ID/EX、EX/MEM、MEM/WB 四组流水线寄存器分别保存哪些数据和控制信号；

- 分析为什么控制信号也必须随数据一起在流水线中传递；
- 给出仿真波形，并标出至少一条指令从 IF 到 WB 的完整流动过程；
- 使用调试读端口展示 R4、R5、R6、R0 和 Mem[4] 的最终结果；
- 说明与单周期数据通路相比，流水线数据通路的主要区别；
- 简要说明为什么题目二需要通过插入 NOP 来避免数据冒险。

题目三：数据冒险检测与转发机制设计

在题目二的简化五级流水线基础上，本题要求实现针对 EX 阶段 ALU 操作数的数据转发 (forwarding/bypassing) 和 load-use 冒险检测与暂停机制，使测试数据集中给出的寄存器型 ALU 相关和 LOAD 后使用相关，能够在不手动插入 NOP 的情况下得到正确结果。本题暂不要求处理 STORE 写数据路径的转发，例如 $\text{Mem}[\text{rs1}+\text{imm}]=\text{rs2}$ 中 rs2 来自前一条指令结果的情形。请在自拟测试中避免此类“STORE 写入数据紧邻依赖前一条指令结果”的序列。

本题沿用题目二的全部时序约定（寄存器堆同步写、组合读、写优先；数据存储器同步写、组合读；地址取 $\text{alu_result}[4:0]$ ；R0 恒零）。

冒险与转发模型（务必严格遵守，否则期望结果不可复现）：

- **转发目标：**转发只作用于 EX 阶段的两个 ALU 操作数输入。即在 EX 阶段，根据当前 ID/EX 中的 rs1/rs2，决定 ALU 的两个输入分别来自寄存器堆读出值、EX/MEM 旁路、还是 MEM/WB 旁路。
- **两条转发路径：**
 1. **EX/MEM $\rightarrow$ EX：**当 EX/MEM 寄存器中的指令将写回寄存器（ALU 结果，即 $\text{op_type}=\text{ALU}$ 且 $\text{rd}\neq 0$ ）且其 rd 等于当前 EX 阶段所需的源寄存器号时，转发 EX/MEM 中的 ALU 结果。
 2. **MEM/WB $\rightarrow$ EX：**当 MEM/WB 寄存器中的指令将写回寄存器（ALU 结果或 LOAD 数据，且 $\text{rd}\neq 0$ ）且其 rd 等于当前 EX 阶段所需的源寄存器号时，转发 MEM/WB 中的写回数据。
- **优先级：**当 EX/MEM 和 MEM/WB 同时命中同一个源寄存器时，优先采用 EX/MEM（更新、距离更近的结果）。
- **R0 例外：**源寄存器号为 0 时不进行转发，恒读 0。
- **load-use 冒险：**转发无法解决“紧邻的下一条指令使用 LOAD 结果”的情形，因为 LOAD 数据在 MEM 阶段末尾才就绪，而紧邻指令在 EX 阶段就需要它。此时必须暂停：
 - **检测条件（在 ID 阶段进行）：**若 ID/EX 寄存器中的指令是 LOAD ($\text{op_type}=\text{LOAD}$, $\text{rd}\neq 0$)，且其 rd 等于当前处于 ID 阶段指令的 rs1 或 rs2，则判定为 load-use 冒险；

- **处理动作**：将 `stall` 置 1 持续一个周期。由于本题没有显式 PC 与取指模块（指令由外部输入），暂停表现为：保持 IF/ID 流水线寄存器不变（对其施加 `stall`），并在 `testbench` 中使外部输入指令保持一个周期；同时向 ID/EX 寄存器插入一个气泡（对 ID/EX 施加 `flush`，使其 `op_type` 清零为 NOP）；
- **检测策略说明**：为降低实现复杂度，本题允许采用**保守检测策略**，即默认除 NOP 外的当前指令都可能使用 `rs1` 和 `rs2`。因此，只要前一条 LOAD 的目的寄存器与当前指令的 `rs1` 或 `rs2` 相同即暂停一拍。该策略可能产生少量不必要的暂停，但不影响功能正确性；
- **暂停后取数来源**：暂停一个周期后，原 LOAD 指令前进至 MEM/WB，相关指令重新进入 EX 阶段，此时通过 **MEM/WB → EX 转发** 获得 LOAD 读出的数据。换言之，load-use 的正确性 = 暂停一拍 + MEM/WB 转发，二者缺一不可。

- 本题复用题目一的 `pipe_reg`：暂停对应 `stall`，插气泡对应 `flush`。

题目三要求在题目二顶层接口（`simple_pipeline_cpu`）基础上新增如下调试输出（用于在波形中验证转发/暂停行为）：

```

1  output wire [1:0] debug_forward_a, // EX 第一操作数转发选择
2  output wire [1:0] debug_forward_b, // EX 第二操作数转发选择
3  output wire      debug_stall      // load-use 暂停信号

```

Listing 3: 题目三要求新增的调试端口

其中转发选择编码建议为：2'b00 取寄存器堆读出值，2'b01 取 EX/MEM 旁路，2'b10 取 MEM/WB 旁路。

各模块功能要求如下：

1. 转发单元 (Forwarding Unit)

- 输入：EX/MEM 与 MEM/WB 阶段的 `rd`、`op_type`（或等价的“是否写寄存器”标志）、当前 ID/EX 阶段的 `rs1` 与 `rs2`；
- 输出：`forward_a`、`forward_b` 两个选择信号，分别控制 EX 阶段两个 ALU 操作数的来源；
- 必须实现 EX/MEM 优先于 MEM/WB 的优先级；
- 源寄存器为 R0 时不转发。

2. 冒险检测单元 (Hazard Detection Unit)

- 输入：ID/EX 阶段的 `op_type` 与 `rd`、IF/ID 阶段的 `rs1` 与 `rs2`；
- 输出：`stall` 信号；
- 仅在 ID/EX 为 LOAD 且其 `rd` 命中 IF/ID 的源寄存器时置 1；
- `stall=1` 时保持 IF/ID 流水线寄存器不变（并在 testbench 中使外部输入指令保持一拍），同时对 ID/EX 施加 `flush` 插入气泡。

测试要求如下：

请在 testbench 中至少完成下文“题目三测试数据集”给出的四组测试。各测试中相关指令紧邻排列、不再手动插入 NOP（末尾的 NOP 仅用于排空流水线）。请通过波形验证：

1. 转发确实发生（观察 `forward_a/forward_b` 在相应周期取非零值）；
2. EX/MEM 优先级正确（测试二）；
3. load-use 时 `stall` 恰好持续一个周期，且 ID/EX 被插入一个气泡（测试三）；
4. 无相关序列下 `stall` 恒为 0、转发选择不误触发（测试四）；
5. 各测试的最终寄存器值与期望一致。

报告要求如下：

- 画出加入转发单元和冒险检测单元后的数据通路图，标出两条转发路径和暂停控制路径；
- 给出转发单元和冒险检测单元的逻辑表达式或真值表；
- 对四组测试分别给出周期-阶段表，并标注每个转发实际发生的周期与来源路径；
- 针对测试三，画出 load-use 冒险的周期-阶段表，标出 `stall=1` 的周期、气泡所在位置，以及相关指令在 EX 阶段经 MEM/WB 取得 LOAD 数据的时刻；
- 说明为什么 EX/MEM 转发优先级要高于 MEM/WB；
- 说明为什么转发无法消除 load-use 冒险，必须配合一拍暂停；
- 对比题目二（手动插 NOP）与题目三（自动转发/暂停）在 IPC (Instruction-per-cycle) 上的差异。

本次作业的测试数据集与验证要求

为保证不同同学的仿真结果具有可比性，本次作业要求至少使用以下测试数据集进行验证。学生可以在此基础上增加更多测试用例，但不得减少以下基本测试。

题目一测试数据集：流水线寄存器

请使用如下输入序列测试 `pipe_reg` 模块。假设 `WIDTH = 8`。该序列经过设计，使得暂停保持、清空气泡和 `flush` 优先三种行为在波形上均可被唯一区分（即每次清零前 `dout` 均为非零值）。

周期	rst_n	stall	flush	din	期望 dout（本拍上升沿后）
0	0	0	0	8'hAA	8'h00（异步复位）
1	1	0	0	8'h11	8'h11（正常传递）
2	1	0	0	8'h22	8'h22（正常传递）
3	1	1	0	8'h33	8'h22（stall 保持）
4	1	1	0	8'h44	8'h22（stall 继续保持）
5	1	0	0	8'h55	8'h55（恢复正常传递）
6	1	0	1	8'h66	8'h00（flush 清零，前值为 0x55）
7	1	0	0	8'h77	8'h77（正常传递，重新装入非零值）
8	1	1	1	8'h88	8'h00（flush 优先，前值 0x77 被清零而非保持）
9	1	0	0	8'h99	8'h99（正常传递，确认已退出 flush）

期望行为如下：

- 复位后，`dout` 应为 `8'h00`；
- 正常传递时，`dout` 应在时钟上升沿更新为 `din`；
- 当 `stall = 1` 时，`dout` 应保持非零原值（周期 3–4 保持 `8'h22`），以区别于清零；
- 当 `flush = 1` 时，`dout` 应清零（周期 6 由 `8'h55` 清为 `8'h00`）；
- 当 `stall = 1` 且 `flush = 1` 时（周期 8），`flush` 优先：`dout` 应由 `8'h77` 清为 `8'h00`，而非保持 `8'h77`。正是因为前一拍 `dout` 为非零值，才能据此判定生效的是 `flush` 而非 `stall`；
- 周期 9 恢复正常传递并装入 `8'h99`，用于确认模块已正确退出 `flush` 状态。

题目二测试数据集：简化五级流水线

题目二要求使用以下寄存器初始值：

寄存器	初始值
R1	16'h0004
R2	16'h0006
R3	16'h1234
R7	16'h00FF

初始化完成后，按照如下顺序每个周期输入一条简化指令。本序列已显式插入 NOP，确保所有数据相关指令之间间隔足够，无需转发即可得到正确结果：

周期	操作	valid	op_type	rs1	rs2	rd	imm	alu_op	byte_en
1	R4 = R1 + R2	1	2'b01	1	2	4	0	3'b000	2'b00
2	Mem[R1+0] = R3	1	2'b11	1	3	0	0	3'b000	2'b11
3	R5 = Mem[R1+0]	1	2'b10	1	0	5	0	3'b000	2'b00
4	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
5	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
6	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
7	R6 = R4 - R2	1	2'b01	4	2	6	0	3'b001	2'b00
8	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
9	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
10	NOP	0	2'b00	0	0	0	0	3'b000	2'b00
11	NOP	0	2'b00	0	0	0	0	3'b000	2'b00

NOP 间隔说明：第 7 周期的 $R6 = R4 - R2$ 依赖第 1 周期 R4 的写回结果。第 1 周期指令在第 5 周期完成 WB (IF=1, ID=2, EX=3, MEM=4, WB=5)，而第 7 周期指令在第 8 周期进入 ID 阶段读取 R4，此时 R4 早已写回，故无需转发。第 2、3 周期的 STORE 与 LOAD 使用同一地址 Mem[4]，STORE 在第 5 周期写入存储器、LOAD 在第 6 周期读出，写在读之前一拍，结果正确。

题目二的期望结果如下：

检查对象	期望结果
R4	16'h000A
Mem[4]	16'h1234
R5	16'h1234
R6	16'h0004
R0	16'h0000

题目三测试数据集：转发与冒险检测

题目三至少需要完成以下四组测试。各组中相关指令**紧邻排列**，末尾的 NOP 仅用于将流水线排空（保证最后一条有效指令完成 WB）。所有测试均沿用题目二的时序约定。

测试一：ALU 到 ALU 的数据转发 寄存器初始值：R1=16'h0004, R2=16'h0006, R3=16'h0003。

周期	指令	说明
1	R4 = R1 + R2	使用上一条结果，触发 EX/MEM→EX 转发 排空流水线
2	R5 = R4 + R3	
3-6	NOP	

期望结果：R4 = 16'h000A, R5 = 16'h000D。若无转发，R5 可能会使用寄存器堆中的旧 R4 值，从而得到错误结果。波形中应观察到第 2 条指令进入 EX 时 forward_a 选择 EX/MEM 旁路。

测试二：连续相关与 EX/MEM 优先级 寄存器初始值：R1=16'h0001, R2=16'h0002, R3=16'h0004。

周期	指令	说明
1	R4 = R1 + R2	依赖 c1 的 R4 (EX/MEM 转发) 依赖 c2 的 R4；此刻 c1 的旧 R4 在 MEM/WB，c2 的新 R4 在 EX/MEM 排空流水线
2	R4 = R4 + R3	
3	R5 = R4 + R2	
4-7	NOP	

期望结果：R4 = 16'h0007, R5 = 16'h0009。该测试验证：当 EX/MEM 与 MEM/WB 同时命中 R4 时，必须优先采用 EX/MEM（即 c2 写的新值 0x0007），否则 R5 会用到 c1 的旧值 0x0003 而出错。

测试三：Load-use 冒险 寄存器与存储器初始值：R1=16'h0004, R2=16'h0006, Mem[4]=16'h0010。

周期	指令	说明
1	R4 = Mem[R1 + 0]	LOAD
2	R5 = R4 + R2	紧邻使用 LOAD 结果，构成 load-use 冒险
3-7	NOP	排空流水线

期望结果：R4 = 16'h0010, R5 = 16'h0016。该测试中应观察到 `stall=1` 持续一个周期、ID/EX 阶段被插入一个气泡；暂停一拍后，第 2 条指令在 EX 阶段通过 MEM/WB→EX 转发取得 LOAD 数据 0x0010，从而 $R5 = 0x0010 + 0x0006 = 0x0016$ 。由于该气泡使 R5 延后一拍完成 WB，请务必在**最后一条有效指令 (R5) 进入 WB 之后再检查寄存器结果**（本测试建议至少运行 5 个 NOP 周期，即周期 3-7）。

测试四：无相关指令序列 寄存器初始值：R1=16'h0004, R2=16'h0006, R3=16'h0002, R6=16'h0008。

周期	指令	说明
1	R4 = R1 + R2	与上一条无数据相关 排空流水线
2	R5 = R3 + R6	
3-6	NOP	

期望结果：R4 = 16'h000A, R5 = 16'h000A。该测试中不应产生 `stall`；当第二条有效指令 $R5 = R3 + R6$ 进入 EX 阶段时，`forward_a` 和 `forward_b` 不应错误选择转发路径，应为 2'b00。

提交材料说明

- Verilog 代码文件 (.v);
- Testbench 代码文件 (.v);
- 仿真波形截图;
- 实验报告 PDF, 包含设计说明、模块接口说明、测试截图、关键代码解释和仿真结果分析;
- 每一道题均需说明模块功能、输入输出端口含义、测试用例设计和仿真结果是否符合预期。

报告撰写要求

- 每道题需要单独成节;
- 每道题需要给出模块接口说明;
- 每道题需要给出核心设计思路;
- 每道题需要贴出关键仿真波形或终端输出截图;
- 每道题需要解释测试结果, 而不是只粘贴截图;
- 若代码无法通过仿真, 需要说明错误原因和已尝试的修改方法。

评分标准

- 题目一: 30 分;
- 题目二: 40 分;
- 题目三: 30 分;